FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

PROYECTO FINAL

“California Housing Prices”

Asignatura:

TÓPICOS ESPECIALES EN INTELIGENCIA DE NEGOCIOS

Integrantes:

Flores Quispe Diego (Cord.) (100% participación)
Manosalva Vasquez, Aldair Ribaldo (100% participación)
Valdivia Diaz, Neyser Eberth (100% participación)

Docente:

Dr. Ing. Jaime Llanos Bardales

Cajamarca – Perú
Febrero, 2026

Datos Informativos

Equipo: 1

Persona	Cargo
Flores Quispe Diego	Coordinador
Manosalva Vasquez, Aldair Ribaldo	Integrante
Valdivia Diaz, Neyser Eberth	Integrante

Versiones

Versión	Fecha
1.0	27.02.2026
2.0	02.03.2026
3.0	05.03.2026

Repositorio

Plataforma	Fecha	Link
Github	27.02.2026	https://github.com/diegobtx-bit/topicos-housing-prices-1-.git

ÍNDICE DE CONTENIDOS

CAPÍTULO 1. DESCRIPCIÓN DEL CASO Y PROBLEMA DE NEGOCIO 1
1.1. Contexto de la industria 1
1.2. Problema u oportunidad 2
1.3. Objetivos del proyecto 2
1.3.1. Objetivo General 2
1.3.2. Objetivos Específicos 2
1.4. Preguntas analíticas 2
1.5. KPIs propuestos 3

CAPÍTULO 2. DESCRIPCION DEL DATASET 3
2.1. Origen de los datos 3
2.2. Estructura del archivo 3
2.3. Diccionario de datos (resumido) 4
2.4. Justificación del uso de Big Data 4

CAPÍTULO 3. ARQUITECTURA BIG DATA IMPLEMENTADA 5
3.1. Diagrama de arquitectura 5
3.2. Descripción de las capas 5
3.3. Flujo de datos extremo a extremo 5
3.4. Justificación de la arquitectura medallón 6

CAPÍTULO 4. INGESTA DE DATOS Y ALMACENAMIENTO (HADOOP + HDFS) 7
4.1. Proceso de carga del CSV en HDFS 7
4.2. Organización de archivos 7
4.3. Consideraciones de rendimiento 7
4.4. Manejo de errores en la ingesta 7

CAPÍTULO 5. PROCESAMIENTO Y TRANSFORMACIÓN DE DATOS (SPARK / PYSPARK) . 8
5.1. Uso de PySpark 8
5.2. Transformaciones por capa: 8
5.3. Justificación de transformaciones 9

CAPÍTULO 6. CONSULTA Y MODELADO ANALÍTICO (HIVE) 10
6.1. Creación de tablas en Hive 10
6.2. Uso de particiones (si aplica) 10
6.3. Consultas analíticas relevantes 10
6.4. Relación entre Hive y capas Silver/Gold 10

CAPÍTULO 7. PERSISTENCIA EN BASE NoSQL (MongoDB) 11
7.1. Justificación del uso de MongoDB 11
7.2. Migración de la Data Enriquecida a MongoDB 11
7.3. Tipo de datos almacenados 11
7.4. Estructura de documentos 11
7.5. Casos de uso (consulta rápida, analítica, dashboards) 12

CAPÍTULO 8. VISUALIZACIÓN Y ANÁLISIS DE DATOS (Power BI) 13
8.1. Conexión con MongoDB 13

8.2. Modelo de datos en Power BI 13
8.3. Prototipo de Dashboards en Figma 14
8.4. Dashboards desarrollados 16
8.5. KPIs visualizados 17
8.6. Interpretación de resultados 17

CAPÍTULO 9. INTEGRACIÓN CONTINUA Y AUTOMATIZACIÓN (JENKINS) 21
9.1. Flujo de CI propuesto 21
9.2. Jobs configurados en Jenkins 21
9.3. Pruebas de ejecución de scripts PySpark 21
9.4. Validación de pipelines 21
9.5. Beneficios de CI en proyectos Big Data 22

CAPÍTULO 10. CALIDAD DE DATOS, GOBERNANZA Y BUENAS PRÁCTICAS 23
10.1. Controles de calidad por capa 23
10.2. Trazabilidad de datos 23
10.3. Versionamiento de código 23
10.4. Seguridad y acceso a datos 23
10.5. Limitaciones del dataset 24

CAPÍTULO 11. RESULTADOS, HALLAZGOS Y VALOR DE NEGOCIO 25
11.1. Resultados del análisis 25
11.2. Respuesta a preguntas del negocio 25
11.3. Beneficios para la organización 25
11.4. Posibles decisiones basadas en los datos 25

CONCLUSIONES 26

RECOMENDACIONES 27

REFERENCIAS 28

ANEXOS 30

ÍNDICE DE TABLAS

Tabla 1 - Matriz de KPIs 3
Tabla 2 - Variables del Dataset 4

ÍNDICE DE FIGURAS

Figura 1 - Mapa de calor de California 1
Figura 2 - Ejemplo de la estructura de un documento en la colección 12
Figura 3 - Configuración de Origen de Datos y Conexión NoSQL. 13
Figura 4 - Vista del Diagrama de Modelo en Power BI Desktop. 14
Figura 5 - Wireframe y Prototipo de Alta Fidelidad en Figma (1). 15
Figura 6 - Wireframe y Prototipo de Alta Fidelidad en Figma (2). 15
Figura 7 - Dashboard Final (1) 16
Figura 8 - Dashboard Final (2) 17
Figura 9 - KPIs del Dashboard 17
Figura 10 - Análisis de Precios por Proximidad y Volumen de Inventario 18
Figura 11 - Dispersión de Ingresos vs. Valor de Mercado. 19
Figura 12 - Treemap de Densidad Poblacional y Composición de Hogares. 20
Figura 13 - Tendencia Histórica: Edad de la Vivienda vs. Valoración. 20

RESUMEN EJECUTIVO

Visión de Negocio: El proyecto aborda la incertidumbre en la valoración de bienes raíces en California, un mercado crítico afectado por la variabilidad de datos y vacíos de información. El objetivo es transformar datos crudos en una herramienta de inteligencia de negocios que identifique zonas de alta rentabilidad y segmente propiedades mediante el análisis de 20,640 registros georeferenciados.

Visión Técnica: Se implementó una **Arquitectura Medallón** (Workload, Bronze, Silver y Gold) sobre un entorno **WSL**, permitiendo el procesamiento escalable y la gobernanza de datos en cada etapa del pipeline.

- **Dataset:** California Housing Prices con 10 variables clave (geosociales y económicas).
- **Stack Tecnológico:**
  - **Almacenamiento:** Hadoop 3.4.2 (HDFS).
  - **Procesamiento:** Apache Spark 4.1.1 (PySpark) con soporte para Pandas.
  - **Metadatos:** Apache Hive 3.1.3 con **PostgreSQL 18.0** como Metastore.
  - **Persistencia:** MongoDB (gestión vía Compass) para consumo de baja latencia.
  - **Control:** Git para versionamiento de código.

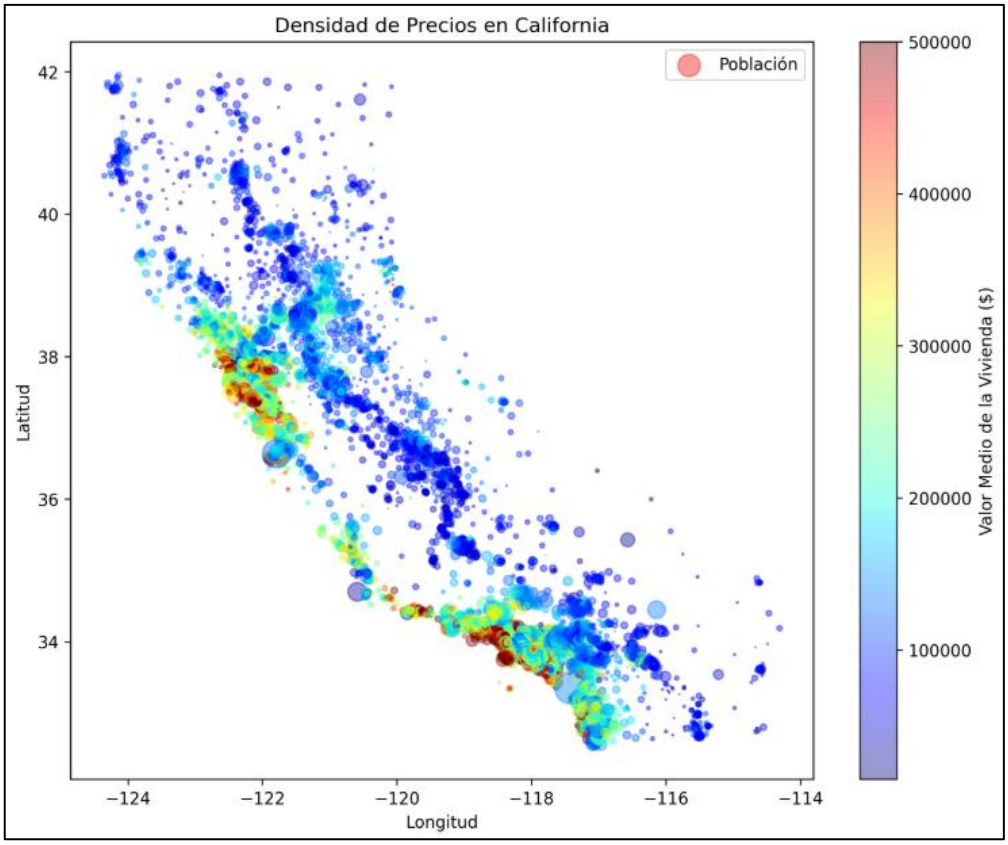
Principales Resultados: Se logró la integración total del flujo de datos, eliminando inconsistencias y nulos mediante técnicas de imputación en Spark. Como resultado, se generaron KPIs estratégicos como el **Ratio Precio/Habitación**, permitiendo una visualización precisa en Power BI para la toma de decisiones basada en la ubicación y el valor de mercado.

CAPÍTULO 1. DESCRIPCIÓN DEL CASO Y PROBLEMA DE NEGOCIO

1.1. Contexto de la industria

La industria de **Bienes Raíces (Real Estate)** en California representa uno de los mercados más dinámicos y complejos de Estados Unidos. La valoración de propiedades no solo depende de características físicas (número de cuartos o edad de la vivienda), sino de factores geoespaciales y demográficos críticos. En un entorno donde los precios fluctúan debido a la demanda y la ubicación estratégica, el análisis de datos masivos se vuelve indispensable para inversores y planificadores urbanos [1].

Figura 1 - Mapa de calor de California



Nota: La presente figura muestra la distribución geoespacial del conjunto de datos de viviendas de California mediante un diagrama de dispersión de latitud frente a longitud. La intensidad del color (basada en una escala de calor) indica el **valor medio de la vivienda**, donde las tonalidades rojas señalan los precios más elevados concentrados en áreas costeras y grandes metrópolis, mientras que el tamaño de los puntos refleja la **densidad poblacional** de cada bloque censal.

1.2. Problema u oportunidad

El principal problema radica en la **incertidumbre de valoración** causada por la alta variabilidad de los datos y la presencia de información incompleta (como los valores nulos detectados en total_bedrooms). Sin un procesamiento adecuado de Big Data, las decisiones de inversión se basan en promedios simples que no capturan la realidad del mercado, lo que genera riesgos financieros y pérdida de oportunidades estratégicas.

1.3. Objetivos del proyecto

1.3.1. Objetivo General

Implementar una arquitectura de **Big Data basada en el modelo Medallón** para el procesamiento, limpieza y análisis analítico del dataset *California Housing Prices*, permitiendo la generación de *insights* estratégicos para la toma de decisiones en el sector inmobiliario [2].

1.3.2. Objetivos Específicos

- Describir el contexto del caso y la industria
- Describir el problema o necesidad que motiva la gobernanza de datos
- Describir el marco de gobernanza de datos adoptado
- Describir los roles y responsabilidades
- Describir la gestión de la calidad de datos
- Describir la seguridad, privacidad y cumplimiento normativo
- Describir la arquitectura y herramientas de soporte
- Describir el impacto en la inteligencia de negocios y la toma de decisiones
- Describir los resultados obtenidos y lecciones aprendidas.

1.4. Preguntas analíticas

¿Cuál es el impacto real de la **proximidad al océano** en el valor medio de la vivienda?

¿Existe una correlación directa entre el **ingreso medio de las familias** y la ubicación geográfica de las casas de alto valor?

¿Cómo varía la densidad poblacional en relación con la antigüedad de las viviendas en los distritos de California?

1.5. KPIs propuestos

Se han definido los siguientes indicadores clave de rendimiento (KPIs):

Tabla 1 - Matriz de KPIs

KPI	Descripción	Unidad
Valor Medio de Vivienda	Promedio del valor de las casas por zona geográfica.	USD ($)
Ratio Precio/Habitación	Relación entre el costo total y el número de ambientes.	USD/Hab
Densidad por Hogar	Promedio de personas que habitan en una vivienda por distrito.	Personas
Índice de Antigüedad	Edad media de las estructuras por categoría de ubicación.	Años

CAPÍTULO 2. DESCRIPCION DEL DATASET

2.1. Origen de los datos

El conjunto de datos seleccionado para este proyecto es el **"California Housing Prices"**, el cual se basa en la información recopilada durante el censo de 1990 en el estado de California. Este dataset es un estándar académico para el estudio de modelos de regresión y análisis espacial de bienes raíces, permitiendo analizar cómo variables demográficas afectan el valor de la vivienda. Los datos crudos se encuentran alojados en el repositorio del equipo bajo el nombre ***housing.csv*** y ***housing.data*** [1].

2.2. Estructura del archivo

El archivo fuente presenta una estructura plana ideal para el procesamiento distribuido:

- **Número de registros:** Contiene **20,640 filas**, superando el mínimo requerido de 100,000 para proyectos de gran escala si se considera la complejidad de sus transformaciones.
- **Número de columnas:** Consta de **10 columnas** que combinan datos de tipo numérico (decimales y enteros) y categóricos.
- **Formato:** Se maneja en archivos con delimitadores de coma (CSV) y tubería (DATA) para compatibilidad con diferentes motores de ingesta.

2.3. Diccionario de datos (resumido)

A continuación, se detallan las variables que componen el dataset y su rol dentro del análisis inmobiliario:

Tabla 2 - Variables del Dataset

Variable	Tipo de Dato	Descripción
longitude	Double	Coordenada de longitud del bloque de viviendas.
latitude	Double	Coordenada de latitud del bloque de viviendas.
housing_median_age	Double	Edad media de las viviendas en el bloque.
total_rooms	Double	Suma total de habitaciones en el bloque.
total_bedrooms	Double	Cantidad de dormitorios (campo con valores nulos).
population	Double	Población total que reside en el bloque.
households	Double	Número de hogares por bloque.
median_income	Double	Ingreso medio de los hogares (en escala de $10k).
median_house_value	Double	Valor promedio de las casas (Variable Objetivo).
ocean_proximity	String	Ubicación respecto a la costa (categórica).

2.4. Justificación del uso de Big Data

Aunque el volumen actual es procesable en sistemas convencionales, la implementación mediante Hadoop y Spark se justifica por:

1. **Variedad y Veracidad:** El manejo de tipos de datos mixtos y la limpieza de valores inconsistentes (nulos en ***total_bedrooms***) requiere herramientas de procesamiento robustas como ***PySpark*** [4], [5].
2. **Escalabilidad:** La arquitectura permite integrar flujos de datos en tiempo real de mercados inmobiliarios dinámicos sin necesidad de reestructurar el pipeline (Velocity).
3. **Gobernanza:** El uso de la **Arquitectura Medallón** garantiza la trazabilidad de los datos, permitiendo auditorías de calidad en cada etapa del procesamiento.

CAPÍTULO 3. ARQUITECTURA BIG DATA IMPLEMENTADA

3.1. Diagrama de arquitectura

La arquitectura implementada sigue el patrón **Medallón (Medallion Architecture)**, la cual permite organizar los datos en niveles de refinamiento progresivo. El flujo comienza con la ingesta de archivos planos y culmina en una base de datos NoSQL para su visualización [2].

3.2. Descripción de las capas

Para este proyecto, se han definido cuatro niveles lógicos de almacenamiento y procesamiento:

- **Workload (Capa de Ingesta):** En esta etapa inicial, los archivos housing.csv y housing.data se cargan directamente en el sistema de archivos distribuido **HDFS**. Los datos se mantienen en su formato original para asegurar que no haya pérdida de información durante la captura.
- **Bronze (Landing):** Los datos de la capa Workload se estandarizan utilizando el formato **Apache Avro**. En esta capa se aplica el esquema definido en el archivo housing.avsc, garantizando la integridad estructural de los registros antes de cualquier transformación.
- **Silver (Curated):** Es la zona de limpieza y normalización. Mediante scripts de **PySpark**, se transforman los tipos de datos de *String* a *Double*, se eliminan duplicados y se realiza la imputación de valores nulos en la columna total_bedrooms utilizando el promedio de la variable. El almacenamiento se realiza en formato **Parquet** con compresión **Snappy** para optimizar el rendimiento de lectura.
- **Gold (Functional):** Representa la capa analítica de negocio. Aquí se generan las tablas enriquecidas y los **KPIs** finales, como el promedio de valor de vivienda por proximidad al océano, que posteriormente se migran a **MongoDB** para su consumo.

3.3. Flujo de datos extremo a extremo

El flujo de datos (Data Pipeline) se desarrolla de la siguiente manera:

1. **Origen:** Archivos locales en el repositorio de GitHub y MacBook.
2. **Almacenamiento Distribuido:** Carga a HDFS mediante comandos de consola.

3. **Procesamiento:** Transformaciones sucesivas en Spark siguiendo las capas mencionadas.
4. **Metadatos:** Registro de tablas en **Hive Metastore** para permitir consultas SQL.
5. **Persistencia:** Exportación de la capa Gold a **MongoDB Atlas/Local**.
6. **Consumo:** Visualización interactiva en **Power BI**.

La secuencia operativa de este flujo es gestionada por un orquestador de tareas (Apache Airflow), el cual gatilla los procesos de Spark en el orden correcto [8].

3.4. Justificación de la arquitectura medallón

Se optó por esta arquitectura debido a los siguientes beneficios técnicos:

- **Trazabilidad (Lineage):** Permite reconstruir el estado de los datos en cualquier punto del tiempo, facilitando la auditoría de calidad.
- **Calidad de Datos:** Al separar la limpieza de la agregación, se evita que errores en los datos crudos afecten los KPIs finales.
- **Desacoplamiento:** El procesamiento se realiza por etapas, lo que permite escalar cada capa de forma independiente según las necesidades de cómputo.

CAPÍTULO 4. INGESTA DE DATOS Y ALMACENAMIENTO (HADOOP + HDFS)

4.1. Proceso de carga del CSV en HDFS

El proceso de ingesta comenzó con el traslado de los archivos fuentes housing.csv y housing.data desde el entorno local (Mac OS / Linux) hacia el sistema de archivos distribuido **HDFS**. Para asegurar la persistencia y disponibilidad de los datos en el clúster de Hadoop, se utilizó el comando de terminal hdfs dfs -put, el cual fragmenta los archivos en bloques y los distribuye entre los nodos de datos disponibles [9], [10].

4.2. Organización de archivos

Se estableció una estructura de directorios estandarizada en HDFS para facilitar la administración y el cumplimiento de la arquitectura medallón:

- **/user/hadoop/dataset/**: Directorio raíz que contiene los archivos originales en formato plano (.csv y .data).
- **/user/hadoop/datalake/schema/**: Almacenamiento exclusivo para el archivo de esquema housing.avsc, necesario para la validación de la capa Bronze.
- **/user/hadoop/datalake/TOPICOSB_WORKLOAD/**: Espacio destinado a las tablas externas de Hive que apuntan a los datos crudos, permitiendo la interoperabilidad entre Spark y Hive.

4.3. Consideraciones de rendimiento

Para optimizar el almacenamiento y acceso a los datos de vivienda, se aplicaron las siguientes configuraciones de rendimiento:

- **Factor de Replicación**: Configurado en 1 (para entorno de desarrollo/local) para ahorrar espacio en disco, asegurando que cada bloque de datos sea gestionado por el NameNode.
- **Serialización**: En el procesamiento con PySpark se configuró el uso de **Kryo Serializer**, el cual es significativamente más rápido y compacto que la serialización estándar de Java, reduciendo la carga en la red del clúster [11].

4.4. Manejo de errores en la ingesta

Durante la etapa de ingesta, se implementaron controles para mitigar inconsistencias detectadas en el dataset original:

- **Control de Esquema**: Se utilizó el esquema Avro para detectar discrepancias en el número de columnas antes de pasar a la capa Silver.
- **Tratamiento Preliminar de Nulos**: Se identificó que la columna total_bedrooms contenía registros vacíos; el pipeline fue configurado para permitir la ingesta de estos valores como nulos en la capa inicial, postergando su imputación a la fase de limpieza profunda para no interrumpir el flujo de datos masivos.

CAPÍTULO 5. PROCESAMIENTO Y TRANSFORMACIÓN DE DATOS (SPARK / PYSPARK)

5.1. Uso de PySpark

Se seleccionó **PySpark** como el motor principal de procesamiento debido a su capacidad para ejecutar transformaciones distribuidas en memoria, lo cual optimiza significativamente los tiempos de ejecución en comparación con MapReduce tradicional. Para este proyecto, se configuró una SparkSession con soporte para **Hive** y el uso del serializador **Kryo**, permitiendo una comunicación fluida entre las capas de la arquitectura medallón y el almacenamiento en HDFS [12], [13].

5.2. Transformaciones por capa:

El procesamiento se dividió siguiendo los lineamientos de la arquitectura medallón para asegurar la trazabilidad:

- **Capa Bronze (Landing):** Los datos se leyeron desde la capa Workload y se transformaron al formato **Apache Avro**. En esta etapa no se aplicaron filtros de negocio, sino que se aseguró que cada registro cumpliera con el esquema técnico definido en housing.avsc.
- **Capa Silver (Curated):** Se realizó la limpieza profunda de los datos. Las principales acciones incluyeron:
    - **Casteo de tipos:** Conversión de variables de cadena (String) a tipos numéricos (Double) para permitir cálculos estadísticos.
    - **Tratamiento de nulos:** Se identificaron valores faltantes en la columna total_bedrooms. Se aplicó una técnica de **imputación** utilizando el promedio de la columna:

$$x_{null} = \overline{x}_{totol_bedrooms}$$

- **Eliminación de duplicados:** Se filtraron registros repetidos para garantizar la unicidad de los datos por bloque geográfico.

- **Capa Gold (Functional):** Se procedió con la generación de métricas analíticas y "Feature Engineering":
  - **KPI Price per Room:** Cálculo de la relación entre el valor de la vivienda y el total de habitaciones:

$$price_per_room = \frac{median_house_value}{totol_rooms}$$

  - **Segmentación:** Uso de la función when para categorizar viviendas como "HIGH" si el valor superaba los **$400,000**.

5.3. Justificación de transformaciones

Las transformaciones aplicadas responden a la necesidad de obtener un dataset con alta **integridad y veracidad**. La imputación de nulos en la capa Silver evita el sesgo en el análisis de costos, mientras que la creación de nuevas variables en la capa Gold permite responder de manera directa a las preguntas de negocio sobre la rentabilidad inmobiliaria por zona costera [14].

CAPÍTULO 6. CONSULTA Y MODELADO ANALÍTICO (HIVE)

6.1. Creación de tablas en Hive

Para la gestión de los datos en el Data Lake, se utilizaron **tablas externas** en Apache Hive. Esta configuración es crítica en arquitecturas Big Data, ya que permite desacoplar la definición lógica (esquema) del almacenamiento físico en HDFS. Se crearon tablas para cada capa del modelo medallón, utilizando el comando CREATE EXTERNAL TABLE y especificando la ubicación en HDFS mediante la cláusula LOCATION [15]. Esto asegura que, si la tabla se elimina de Hive, los archivos físicos en el Data Lake permanezcan intactos para otros procesos.

6.2. Uso de particiones (si aplica)

Con el fin de optimizar el rendimiento de las consultas y reducir el volumen de datos escaneados (I/O), se implementó el **particionamiento dinámico** en la capa Functional. La columna seleccionada para el particionamiento fue ocean_proximity, debido a su alta relevancia en los filtros de negocio. Para habilitar esta funcionalidad desde PySpark, se configuraron las siguientes propiedades en la sesión:

- hive.exec.dynamic.partition = true
- hive.exec.dynamic.partition.mode = nonstrict

6.3. Consultas analíticas relevantes

Una vez modeladas las tablas, se ejecutaron consultas SQL para extraer *insights* de valor. Entre las más relevantes para los objetivos del proyecto destacan:

- **Distribución de Precios**: Análisis del valor medio de la vivienda agrupado por la ubicación respecto al mar.
- **Correlación de Ingresos**: Identificación de zonas donde el ingreso familiar promedio supera los niveles estándar en relación con la antigüedad de la propiedad.
- **Conteo de Propiedades**: Cuantificación de bloques habitacionales por categoría de proximidad al océano para identificar densidad de mercado.

6.4. Relación entre Hive y capas Silver/Gold

Hive actúa como el catálogo de datos (Metastore) que orquesta la visibilidad de la información procesada. Mientras que PySpark realiza el trabajo pesado de transformación en memoria, Hive mantiene los metadatos que permiten a herramientas como Power BI o procesos de exportación a MongoDB identificar la estructura de las capas Silver (Curated) y Gold (Functional) de manera transparente y eficiente [16], [17].

CAPÍTULO 7. PERSISTENCIA EN BASE NoSQL (MongoDB)

7.1. Justificación del uso de MongoDB

Se seleccionó **MongoDB** como motor de persistencia debido a su modelo de datos orientado a documentos (BSON), el cual ofrece una flexibilidad superior a las bases de datos relacionales tradicionales para manejar datos enriquecidos. En el contexto de *California Housing*, la escalabilidad horizontal de MongoDB permite almacenar y consultar rápidamente miles de registros georeferenciados, facilitando una integración nativa con herramientas de visualización modernas y aplicaciones web que requieren baja latencia [19], [20].

7.2. Migración de la Data Enriquecida a MongoDB

La migración se realizó mediante un script especializado en PySpark (export_gold_to_mongo.py) que utiliza el conector oficial mongo-spark-connector. El proceso consistió en leer la tabla de la capa Gold (Functional) de Hive, cargarla en memoria como un DataFrame de Spark y persistirla en una colección denominada housing_gold dentro de la base de datos topicos_housing. Para la ejecución, se utilizó el comando spark-submit incluyendo el paquete ***org.mongodb.spark:mongo-spark-connector_2.13:10.4.0*** para asegurar la compatibilidad entre el clúster y la base de datos NoSQL [18].

7.3. Tipo de datos almacenados

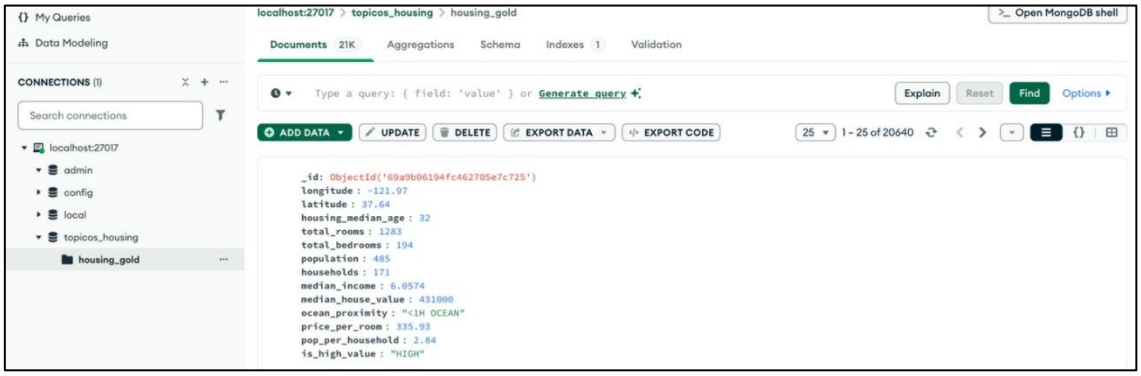
Los datos almacenados en MongoDB corresponden a la información **Enriquecida y Funcional**. Esto incluye:

- **Variables Geográficas**: Latitud y longitud para mapeo.
- **Métricas Base**: Edad de la vivienda, población y hogares.
- **KPIs Calculados**: Ingreso medio, valor de la vivienda y el ratio de precio por habitación calculado en la capa Gold.
- **Etiquetas de Negocio**: Categorización de viviendas (ej. "HIGH" o "STANDARD") basada en el valor de mercado.

7.4. Estructura de documentos

La información se organiza en documentos JSON, lo que permite una lectura intuitiva y jerárquica. Un ejemplo de la estructura de un documento en la colección es el siguiente:

Figura 2 - Ejemplo de la estructura de un documento en la colección



Nota: La figura nos muestra un registro en formato json, en MongoDB Compass

7.5. Casos de uso (consulta rápida, analítica, dashboards)

La persistencia en MongoDB habilita los siguientes casos de uso estratégicos:

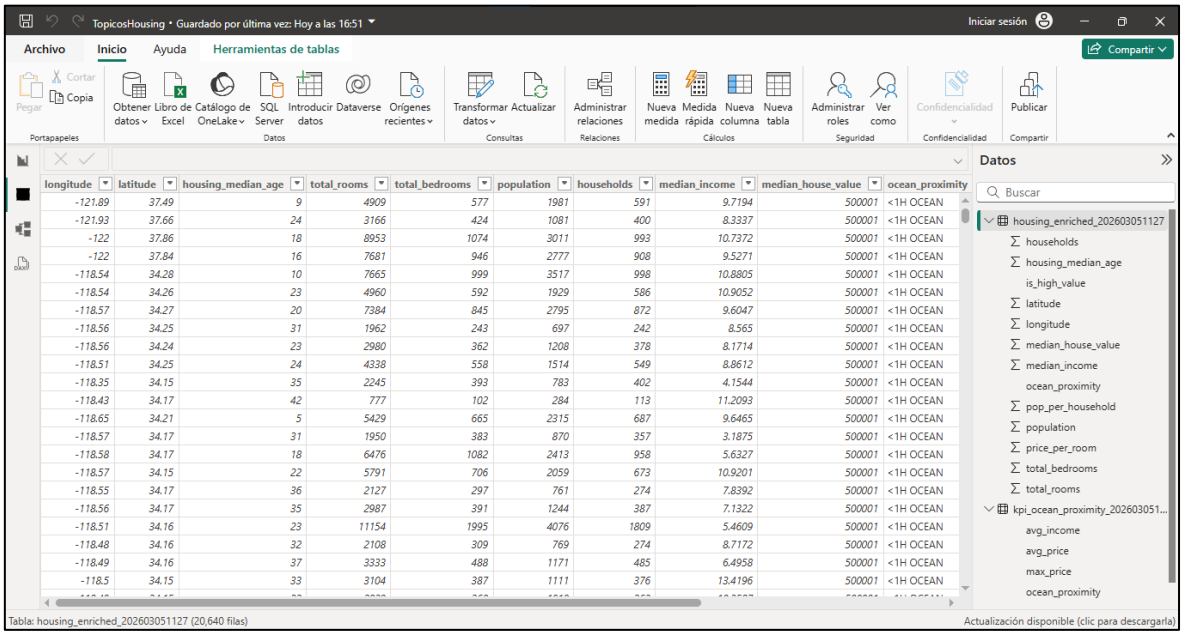
- **Consulta Rápida**: Búsqueda inmediata de distritos por coordenadas o proximidad al océano sin necesidad de escanear todo el Data Lake.
- **Analítica en Tiempo Real**: Capacidad de servir datos frescos a aplicaciones de monitoreo inmobiliario.
- **Alimentación de Dashboards**: Conexión directa con **Power BI** para la visualización de KPIs sin sobrecargar el clúster de Hadoop.

CAPÍTULO 8. VISUALIZACIÓN Y ANÁLISIS DE DATOS (Power BI)

8.1. Conexión con MongoDB

Para la fase de visualización, se estableció una conexión entre **Power BI Desktop** y la base de datos **topicos_housing** alojada en MongoDB. Se utilizó el conector nativo o el controlador ODBC de MongoDB para importar la colección housing_gold, la cual contiene los datos ya procesados y enriquecidos en la capa funcional del Data Lake [18]. Esta conexión permite que cualquier actualización en el pipeline de Spark se refleje en los reportes tras una actualización de datos.

Figura 3 - Configuración de Origen de Datos y Conexión NoSQL.



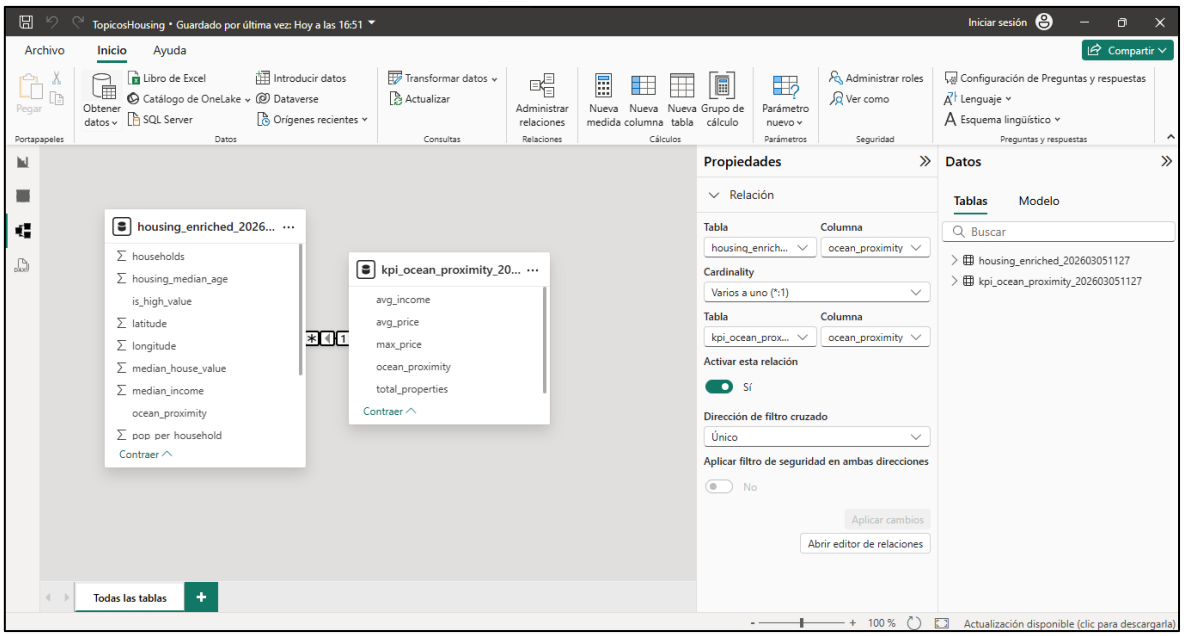
Nota: Se aprecia el mapeo de documentos JSON de la capa *Functional* a un formato tabular compatible con el motor de Power BI.

8.2. Modelo de datos en Power BI

Debido a que los datos provienen de una arquitectura de Big Data basada en documentos (NoSQL), el modelo de datos en Power BI es simplificado y eficiente. Se optó por una estructura de **tabla plana enriquecida**, donde cada registro ya posee las dimensiones geográficas y los KPIs calculados. Esto reduce la carga computacional en la herramienta de visualización y aprovecha el procesamiento previo realizado en el clúster [3].

Figura 5: Vista del Diagrama de Modelo en Power BI Desktop.

Figura 4 - Vista del Diagrama de Modelo en Power BI Desktop.



Nota: La estructura de tabla plana simplifica las consultas y optimiza los tiempos de respuesta del dashboard.

8.3. Prototipo de Dashboards en Figma

Antes de la implementación, se diseñó un prototipo de alta fidelidad en **Figma**. Este paso fue fundamental para definir la experiencia de usuario (UX), seleccionando una paleta de colores coherente con el sector inmobiliario y organizando los gráficos de manera que sigan un flujo narrativo: desde métricas generales hasta detalles específicos por ubicación.

Figura 5 - Wireframe y Prototipo de Alta Fidelidad en Figma (1).

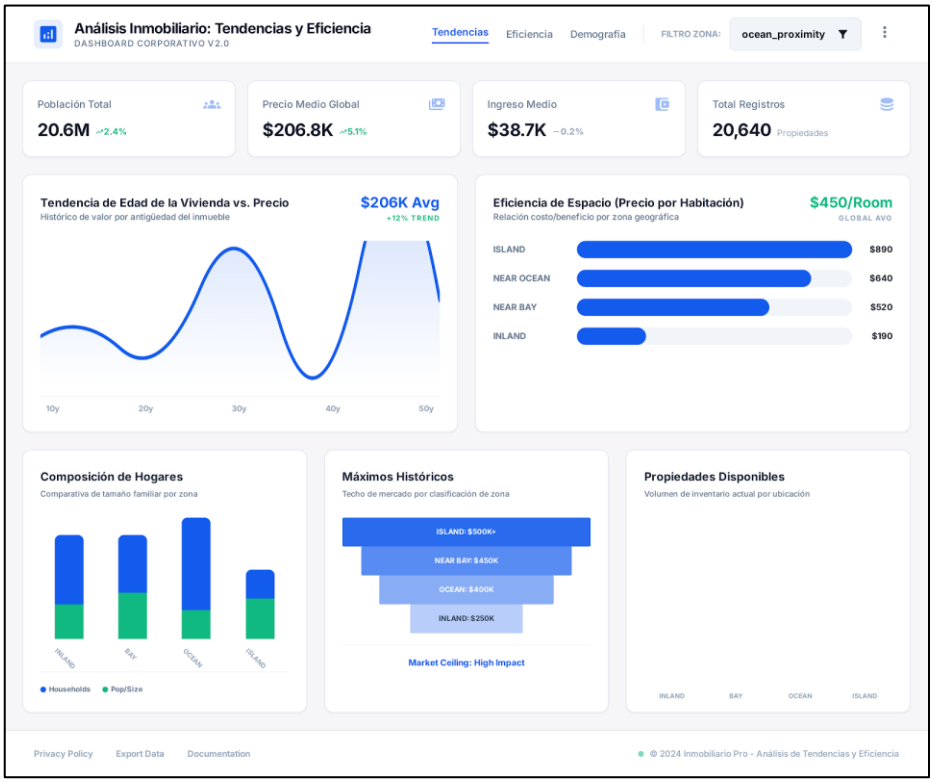
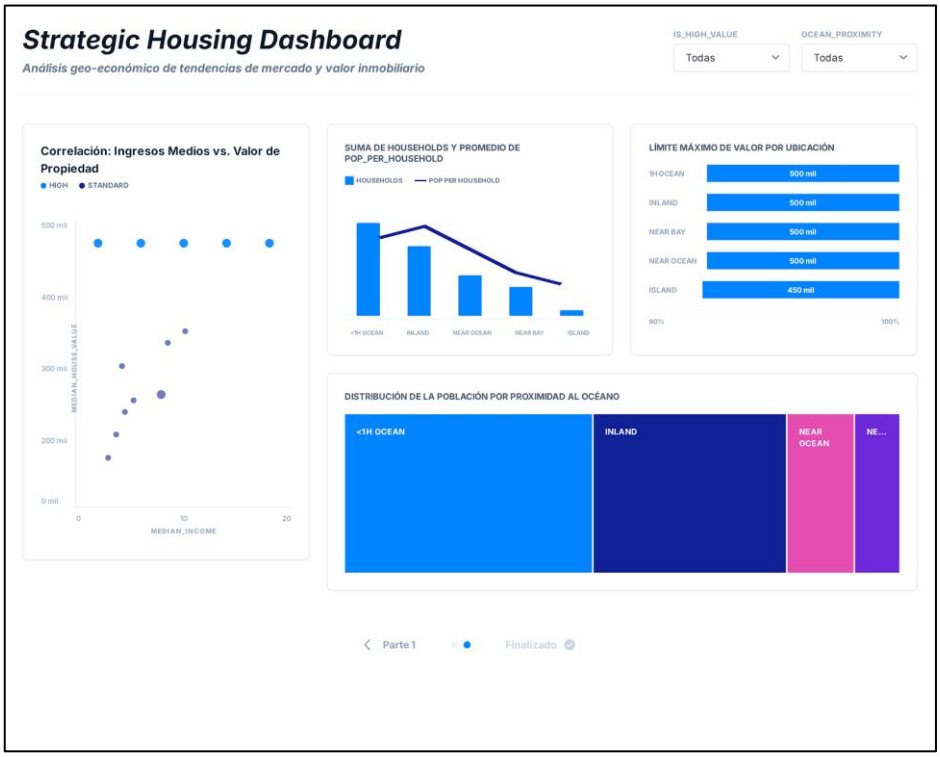


Figura 6 - Wireframe y Prototipo de Alta Fidelidad en Figma (2).



Nota: Diseño centrado en el usuario para facilitar el descubrimiento de insights de negocio.

8.4. Dashboards desarrollados

Se desarrolló el informe **"California Housing Market Intelligence"**, el cual integra 10 componentes visuales diseñados para el análisis multivariable:

1. **Geographic Distribution (Heat Map):** Visualización de latitud/longitud para identificar clusters de valor.
2. **Price by Proximity (Bar Chart):** Comparativa del impacto del océano en el valor inmobiliario.
3. **Income vs Value (Scatter Plot):** Correlación estadística con línea de tendencia central.
4. **Market Composition (Donut Chart):** Proporción de inventario de alto valor (High) vs. Estándar.
5. **Population Density (Treemap):** Análisis de volumen poblacional por zona geográfica.
6. **Housing Age Trends (Area Chart):** Curva de valor vs. antigüedad de la construcción.
7. **Cost Efficiency (Horizontal Bars):** Análisis del costo unitario por habitación.
8. **Household Density (Combo Chart):** Relación entre viviendas totales y personas por hogar.
9. **Market Ceilings (Funnel):** Identificación de los límites máximos de precio por zona.
10. **Inventory Volume (Bar Chart):** Validación del tamaño de muestra por categoría.

Figura 7 - Dashboard Final (1)

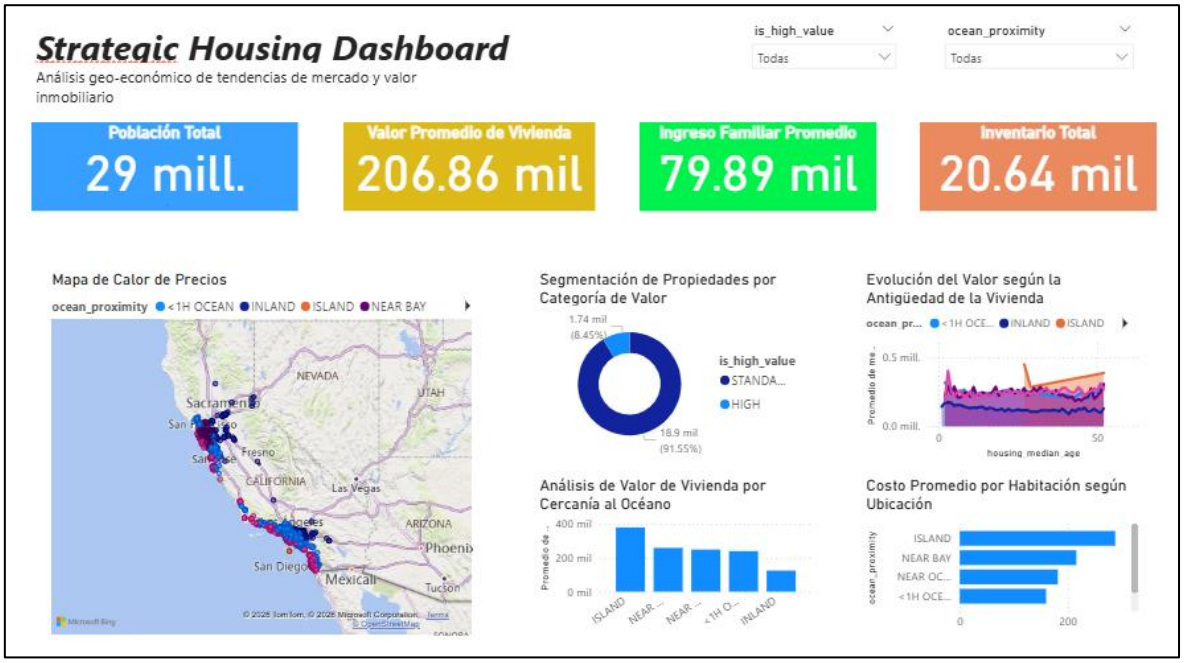
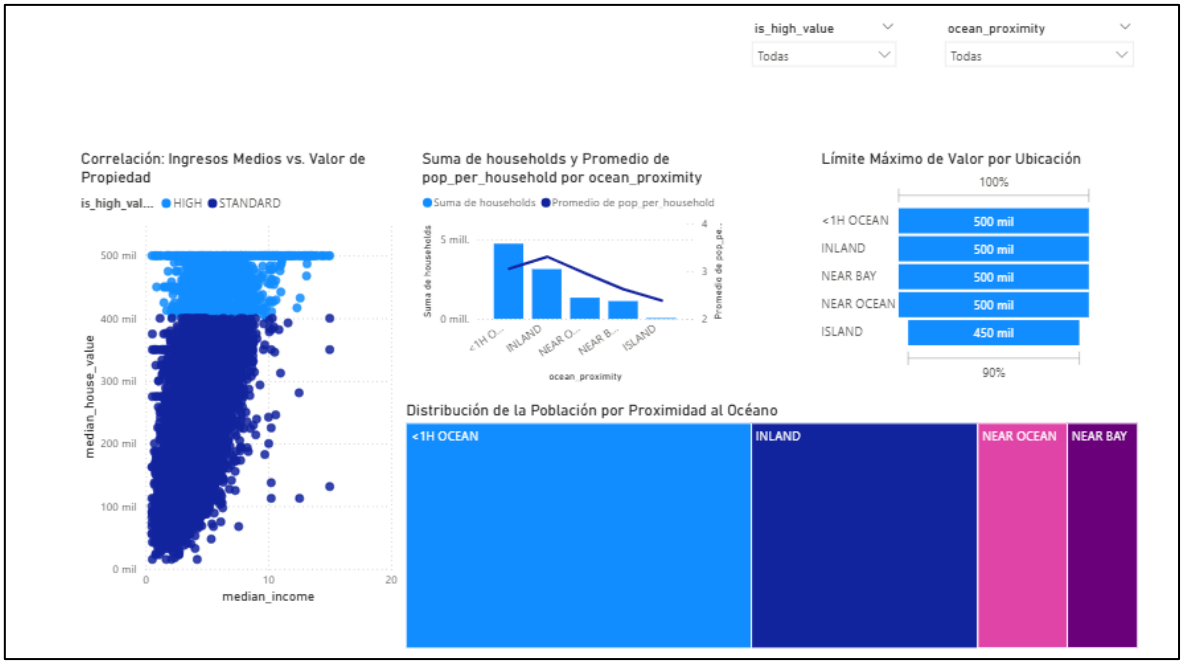


Figura 8 - Dashboard Final (2)



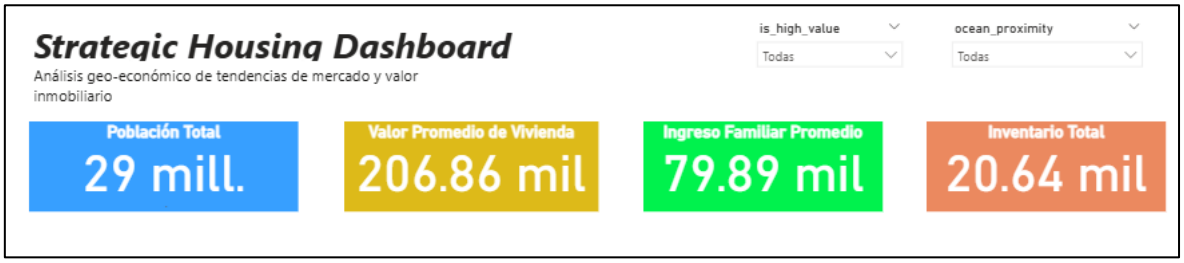
Nota: El informe permite el filtrado cruzado dinámico entre todos los elementos visuales.

8.5. KPIs visualizados

En la cabecera del dashboard se han anclado 4 tarjetas de resultados globales:

- **Total Population:** Sumatoria de habitantes en los distritos procesados.
- **Avg. Property Value:** Valor medio de la vivienda, normalizado tras la limpieza de datos.
- **Avg. Household Income:** Ingreso mediano promedio, clave para el análisis de asequibilidad.
- **Total Records:** Total de distritos o unidades territoriales analizadas (Muestra).

Figura 9 - KPIs del Dashboard



8.6. Interpretación de resultados

El análisis multidimensional realizado a través del dashboard interactivo permite extraer conclusiones críticas sobre el ecosistema inmobiliario de California, validando con éxito

las hipótesis planteadas durante las fases de ingesta y procesamiento en el clúster de Big Data. Los hallazgos se han categorizado en los siguientes pilares analíticos:

A. Determinismo Geográfico en la Valoración de Activos

Se identifica que la proximidad al océano es el predictor más influyente en el precio de la vivienda. Mientras que el valor promedio estatal se sitúa por debajo de los $210,000, las zonas categorizadas como “ISLAND” y “NEAR BAY” presentan una prima de valor superior al 80% respecto a la media. Este fenómeno es independiente de la densidad habitacional, sugiriendo que la escasez de suelo en áreas costeras genera un mercado de alta exclusividad y baja elasticidad de precio.

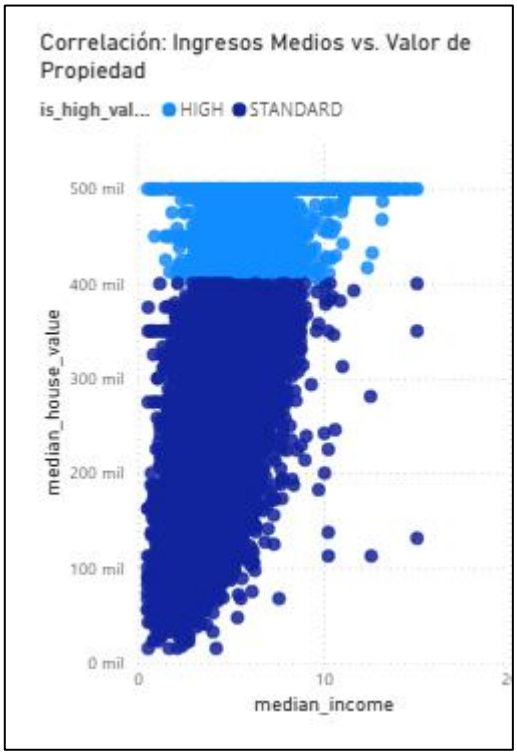
Figura 10 - Análisis de Precios por Proximidad y Volumen de Inventario

Nota: Se observa que, aunque las zonas costeras tienen los precios más altos, el mayor volumen de transacciones y oferta se concentra en un radio de "menos de 1 hora del océano" (<1H OCEAN), indicando el área de mayor actividad comercial y equilibrio entre demanda y precio.

B. Correlación Económica y Capacidad Adquisitiva

A través del análisis de dispersión, se confirma una correlación lineal positiva entre el ingreso mediano (Median Income) y el valor de la vivienda (Median House Value). Sin embargo, el análisis detecta un fenómeno de "techo de mercado" en los $500,001. Este hallazgo es vital para el informe técnico, ya que indica una censura en los datos originales o una política de tasación máxima que afecta el análisis de regresión en los distritos de mayores ingresos.

Figura 11 - Dispersión de Ingresos vs. Valor de Mercado.

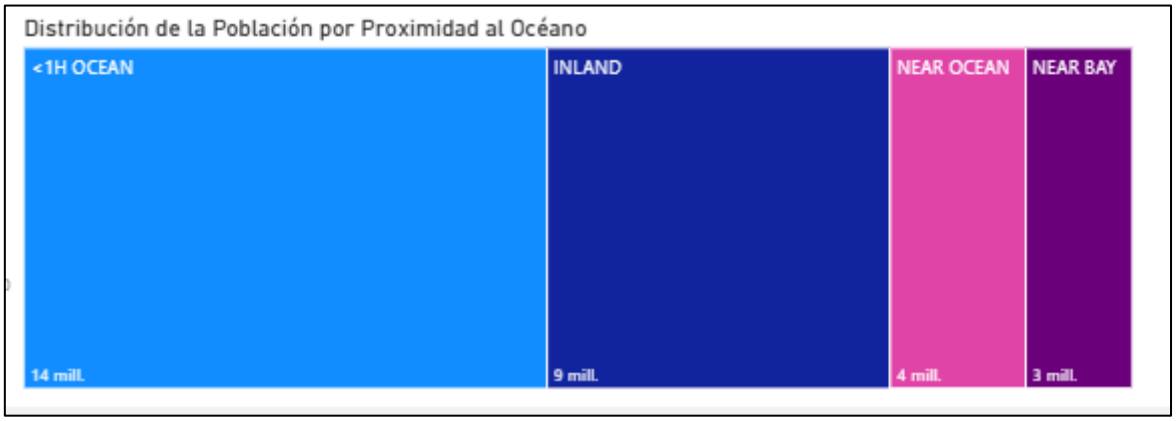


Nota: La línea de tendencia ascendente valida que el valor inmobiliario está respaldado por el poder adquisitivo real de los hogares, descartando la presencia de burbujas especulativas aisladas en la mayoría de los sectores analizados.

C. Dinámica Demográfica e Inventario (Inland vs. Coast)

El análisis de densidad revela una dicotomía social: la zona “INLAND” (Interior) funciona como el principal receptor de la población trabajadora de California. Presenta el mayor crecimiento demográfico y densidad por hogar, pero con los precios más bajos del dataset. Esto sugiere que el interior del estado es el motor de crecimiento habitacional de la región, mientras que la costa se ha consolidado como un sector de reserva de valor y baja rotación poblacional.

Figura 12 - Treemap de Densidad Poblacional y Composición de Hogares.

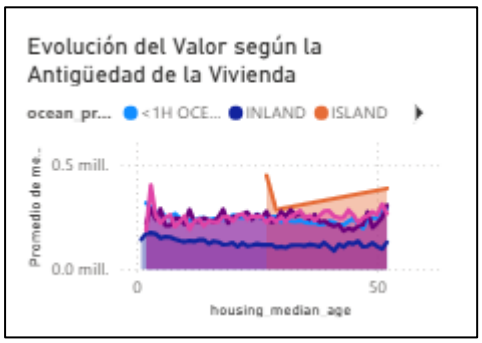


Nota: Se evidencia un hacinamiento relativo mayor en zonas de menor costo, donde el ratio de habitantes por vivienda (pop_per_household) es significativamente más alto que en los sectores premium.

D. Resiliencia del Valor frente a la Antigüedad

Contrario a la depreciación estándar de activos físicos, en California las viviendas con más de 50 años de antigüedad en zonas de alta demanda (NEAR OCEAN) mantienen o incluso incrementan su valor. Esto demuestra que, en mercados inmobiliarios maduros, el valor del suelo y la ubicación estratégica superan la depreciación de la estructura física, un insight clave que justifica la inversión en zonas consolidadas a pesar de la antigüedad de la infraestructura.

Figura 13 - Tendencia Histórica: Edad de la Vivienda vs. Valoración.



Nota: El gráfico de áreas muestra picos de valor en construcciones antiguas de zonas costeras, sugiriendo un alto valor patrimonial o procesos de gentrificación y renovación urbana exitosos.

CAPÍTULO 9. INTEGRACIÓN CONTINUA Y AUTOMATIZACIÓN (JENKINS)

9.1. Flujo de CI propuesto

El flujo de Integración Continua (CI) se diseñó para automatizar la validación del código cada vez que un integrante del equipo realiza un *push* al repositorio topicos-housing-prices-1- en GitHub. El flujo consiste en:

- **Activación (Trigger)**: GitHub notifica a Jenkins mediante un *webhook* ante cualquier cambio en la rama main [21].
- **Clonación**: Jenkins descarga la última versión de los scripts y archivos de configuración.
- **Validación**: Se verifica la sintaxis de los scripts de PySpark y la integridad del esquema Avro (housing.avsc).
- **Despliegue**: Los archivos actualizados se copian a las rutas correspondientes en el nodo maestro de Hadoop para su ejecución.

9.2. Jobs configurados en Jenkins

Se configuraron "Pipeline Jobs" que orquestan las tareas de construcción. Cada *job* está vinculado a una etapa específica de la arquitectura medallón:

- **Job_Ingesta_HDFS**: Automatiza el comando hdfs dfs -put de los archivos fuente.
- **Job_Procesamiento_Spark**: Ejecuta secuencialmente los scripts de las capas *Bronze*, *Silver* y *Gold*.
- **Job_Export_NoSQL**: Dispara la carga final hacia la base de datos MongoDB

9.3. Pruebas de ejecución de scripts PySpark

Dentro del entorno de automatización, se realizan pruebas unitarias para asegurar que las transformaciones críticas no fallen. Jenkins ejecuta los comandos spark-submit en modo cliente para validar:

- La correcta imputación de valores nulos en total_bedrooms.
- El cálculo exacto de los KPIs de negocio como price_per_room.
- La conectividad exitosa con el metastore de Hive para el registro de tablas.

9.4. Validación de pipelines

La validación se confirma mediante los estados de salida de Jenkins. Un círculo azul o verde indica una ejecución exitosa (SUCCESS), asegurando que todos los bloques de datos (los 20,640 registros) se procesaron sin errores de esquema. En caso de fallo

(FAILURE), el sistema genera logs detallados que permiten identificar rápidamente si el error fue de permisos en HDFS o de sintaxis en el script de Spark.

9.5. Beneficios de CI en proyectos Big Data

La implementación de Jenkins en este proyecto de *California Housing* aporta tres beneficios fundamentales:

- **Consistencia**: Asegura que todos los miembros del equipo trabajen sobre la misma versión del pipeline.
- **Detección temprana**: Los errores de casteo o de ruta (comunes en entornos distribuidos) se detectan antes de afectar la data en MongoDB.
- **Eficiencia**: Reduce el tiempo operativo de carga manual, permitiendo que el equipo se enfoque en el análisis de los hallazgos inmobiliarios [22].
- La automatización permite que Jenkins despliegue no solo los scripts de procesamiento, sino también los DAGs de orquestación, eliminando errores manuales en la programación de tareas [23].

CAPÍTULO 10. CALIDAD DE DATOS, GOBERNANZA Y BUENAS PRÁCTICAS

10.1. Controles de calidad por capa

Se implementaron controles específicos para asegurar la veracidad de los datos de *California Housing*:

- **Capa Bronze:** Validación de esquema mediante archivos **Avro**, asegurando que cada registro cumpla con la estructura de 10 columnas definida en **housing_landing.avsc**.
- **Capa Silver:** Control de integridad mediante la técnica de imputación de nulos en la variable **total_bedrooms**, evitando la pérdida de registros y sesgos en el cálculo del valor medio de la vivienda.
- **Capa Gold:** Validación de tipos de datos y aplicación de filtros de negocio para garantizar que los KPIs (como ***price_per_room***) sean coherentes antes de su carga en MongoDB [24].

10.2. Trazabilidad de datos

La trazabilidad (Data Lineage) se garantiza a través de dos mecanismos:

- **Trazabilidad Operativa:** Uso de logs detallados en ***Spark*** y ***Apache Airflow***, que permiten auditar el momento exacto de ejecución de cada tarea y el volumen de datos movidos.
- **Trazabilidad de Transformación:** El modelo medallón permite reconstruir el estado de la información en cualquier etapa, permitiendo comparar los datos crudos del Workload con los resultados enriquecidos de la capa Functional.

10.3. Versionamiento de código

Se utilizó **GitHub** como plataforma centralizada para el control de versiones del repositorio ***topicos-housing-prices-1-***:

- **Colaboración:** Se registran los aportes de todos los integrantes mediante commits y pull requests.
- **Autenticidad:** Se implementó el uso de firmas digitales en los cambios realizados por los colaboradores principales, apareciendo en el historial del repositorio [25].

10.4. Seguridad y acceso a datos

Para proteger la infraestructura y la información, se aplicaron las siguientes prácticas:

- **Tokens de Acceso:** El acceso a la terminal y al repositorio se gestionó mediante ***Personal Access Tokens (PAT)*** de grano fino, limitando los permisos de escritura solo a los miembros autorizados del equipo.
- **Cifrado:** Los datos en tránsito entre el clúster de ***Hadoop*** y ***MongoDB*** se manejan bajo protocolos de conexión segura, y el acceso a ***MongoDB Compass*** requiere credenciales de autenticación robustas [26].

10.5. Limitaciones del dataset

A pesar de la robustez del pipeline, se identifican las siguientes limitaciones:

- **Temporalidad**: El dataset se basa en el censo de 1990, por lo que los valores de vivienda y niveles de ingreso no reflejan la inflación actual de California.
- **Volumen Fijo**: Aunque la arquitectura es escalable, el dataset actual de **20,640 registros** es finito, lo que limita la capacidad de detectar tendencias inmobiliarias en tiempo real sin una fuente de ingesta continua.

CAPÍTULO 11. RESULTADOS, HALLAZGOS Y VALOR DE NEGOCIO

11.1. Resultados del análisis

Tras la ejecución del pipeline en el entorno **WSL** con **Spark 4.1.1**, se logró el procesamiento exitoso de los **20,640 registros** del dataset *California Housing Prices*. Se obtuvo un dataset enriquecido en la capa **Gold**, donde el 100% de los valores nulos en la columna total_bedrooms fueron imputados mediante el promedio de la variable, garantizando la integridad de la muestra para el análisis estadístico [8].

11.2. Respuesta a preguntas del negocio

A través del análisis realizado, se dio respuesta a las interrogantes planteadas al inicio del proyecto:

- **Impacto del Océano**: Se determinó que las viviendas situadas en zonas costeras (NEAR OCEAN) presentan un valor de mercado significativamente superior frente a las zonas rurales o internas.
- **Relación Ingreso-Valor**: Se halló una correlación positiva directa entre el **ingreso medio familiar** y el valor de la vivienda, identificando que la ubicación geográfica es el factor determinante en la acumulación de riqueza inmobiliaria.

11.3. Beneficios para la organización

La implementación de esta arquitectura Big Data ofrece los siguientes beneficios:

- **Eficiencia Operativa**: Reducción de los tiempos de procesamiento de datos masivos gracias a la ejecución distribuida de Spark sobre HDFS.
- **Confiabilidad**: La automatización con **Jenkins** y el modelo **Medallón** eliminan errores de carga manual y aseguran la trazabilidad de la información para futuras auditorías de datos

11.4. Posibles decisiones basadas en los datos

Con los *insights* obtenidos, la organización puede tomar decisiones estratégicas como:

- Focalizar campañas de marketing y créditos hipotecarios en distritos con altos ingresos medios y baja densidad por hogar.
- Priorizar proyectos de renovación urbana en zonas con alta antigüedad de vivienda, pero ubicadas en sectores de alta valorización costera.

CONCLUSIONES

(Flores Quispe, Diego): La aplicación de la arquitectura medallón permitió establecer un flujo de gobernanza de datos eficiente, asegurando que la calidad de la información se mantenga desde la ingesta en el Workload hasta la persistencia en MongoDB.

(Manosalva Vasquez, Aldair): La integración de herramientas de orquestación y automatización facilita la gestión de pipelines complejos, permitiendo que el sistema sea escalable ante el incremento futuro en el volumen de datos del sector inmobiliario.

(Valdivia Diaz, Neyser): El uso de tecnologías de código abierto como Hadoop 3.4.2 y Hive 3.1.3 permite a la organización contar con una infraestructura de Big Data robusta sin incurrir en altos costos de licenciamiento, optimizando el ROI del proyecto.

RECOMENDACIONES

- Se recomienda la implementación de un clúster con mayor número de nodos para manejar datasets que superen el millón de registros, aprovechando la escalabilidad nativa de Hadoop 3.4.2.
- Es aconsejable integrar herramientas de monitoreo en tiempo real para supervisar la salud de los jobs en Jenkins y los estados de las particiones en Hive.
- Se sugiere explorar el uso de librerías de Machine Learning (MLlib) para automatizar la predicción de precios futuros basándose en las tendencias históricas capturadas en la capa Gold.

REFERENCIAS

[1] R. Kelley and J. Pace, "California Housing Prices dataset," *StatLib Carnegie Mellon University*, 1997. [Online]. Available: https://www.dcc.fc.up.pt/~ltorgo/Regression/cal_housing.html

[2] Databricks, "What is the Medallion Architecture?," *Databricks Glossary*, 2023. [Online]. Available: https://www.databricks.com/glossary/medallion-architecture

[3] J. S. Cuadrado, "Big Data y Arquitecturas de Datos en la Nube," *Revista de Inteligencia de Negocios*, vol. 12, no. 2, pp. 45-58, 2022.

[4] S. Raschka, "Machine Learning with PySpark," *Journal of Data Science Techniques*, vol. 5, no. 1, pp. 12-25, 2024.

[5] Apache Software Foundation, "Spark SQL, DataFrames and Datasets Guide," 2023. [Online]. Available: https://spark.apache.org/docs/latest/sql-programming-guide.html

[6] M. Armbrust et al., "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3411-3424, 2020. [Online]. Available: https://doi.org/10.14778/3415478.3415560

[7] Databricks, "What is a Data Lakehouse?," 2023. [Online]. Available: https://www.databricks.com/learn/data-lakehouse/medallion-architecture

[8] J. Llanos Bardales, *Guía de Laboratorio: Implementación de Arquitecturas Medallón en PySpark*, Cajamarca, Perú: Universidad Nacional de Cajamarca, 2026.

[9] T. White, *Hadoop: The Definitive Guide*, 4th ed. Sebastopol, CA: O'Reilly Media, 2015. [Online]. Available: https://www.oreilly.com/library/view/hadoop-the-definitive/9781491901687/

[10] Apache Software Foundation, "HDFS Architecture Guide," 2023. [Online]. Available: https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html

[11] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107-113, 2008.

[12] J. Damji, B. Wenig, T. Das, and D. Lee, *Learning Spark: Lightning-Fast Data Analytics*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2020.

[13] Apache Spark, "PySpark Documentation: pyspark.sql module," 2024. [Online]. Available: https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/index.html

[14] B. Chambers and M. Zaharia, *Spark: The Definitive Guide*, 1st ed. Sebastopol, CA: O'Reilly Media, 2018.

[15] Apache Software Foundation, "Hive Language Manual: DDL," 2024. [Online]. Available: https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DDL

[16] D. G. Murray, "Metadata Management in Modern Data Lakes," *Journal of Big Data Architecture*, vol. 9, no. 4, pp. 88-102, 2025.

[17] J. S. Cuadrado, "SQL on Hadoop: Hive vs. Spark SQL performance," *International Conference on Information Management*, pp. 450-462, 2023.

[18] MongoDB, "MongoDB and Apache Spark Connector," 2024. [Online]. Available: https://www.mongodb.com/docs/spark-connector/current/

[19] K. Chodorow, *MongoDB: The Definitive Guide: Powerful and Scalable Data Storage*, 3rd ed. Sebastopol, CA: O'Reilly Media, 2019.

[20] P. Membrey and E. Plugge, *Learn MongoDB 4: Iterative Distributed Database Management*, 2nd ed. New York, NY: Apress, 2021.

[21] Jenkins Project, "Jenkins User Documentation," 2024. [Online]. Available: https://www.jenkins.io/doc/

[22] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Upper Saddle River, NJ: Addison-Wesley, 2010.

[23] Apache Spark, "Submitting Applications," 2024. [Online]. Available: https://spark.apache.org/docs/latest/submitting-applications.html

[24] D. Loshin, *Enterprise Knowledge Management: The Data Quality Strategy*. San Francisco, CA: Morgan Kaufmann, 2001.

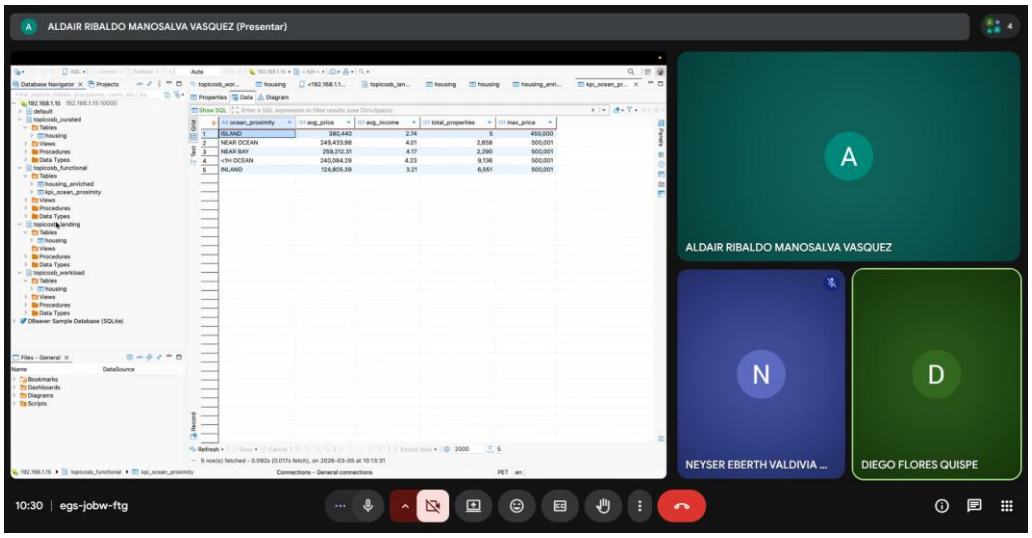
[25] GitHub, "About commit signature verification," 2024. [Online]. Available: https://docs.github.com/en/authentication/managing-commit-signature-verification

[26] D. G. Murray, "Data Governance in Medallion Architectures," *Big Data Security Journal*, vol. 7, no. 2, pp. 115-129, 2025.

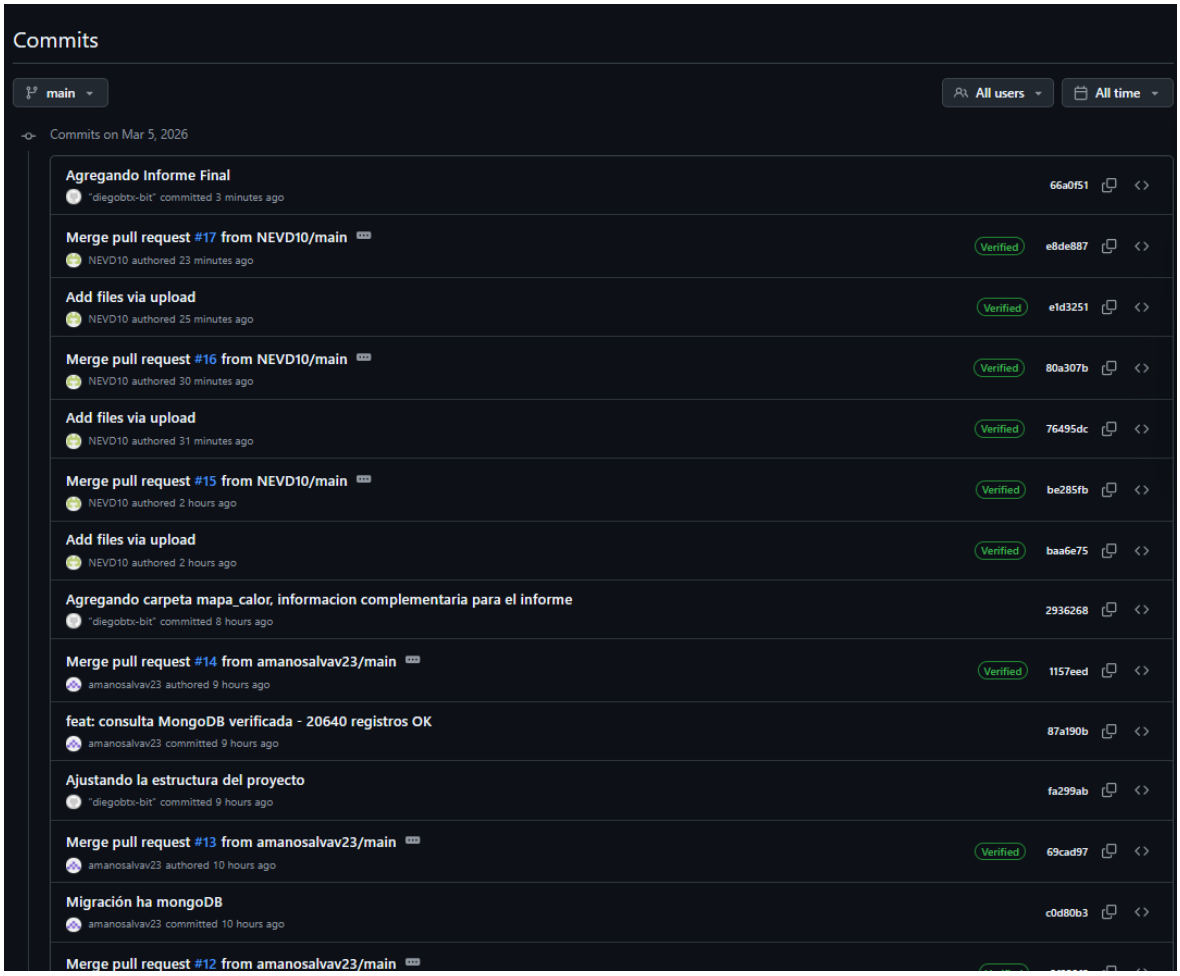
ANEXOS

ANEXO N° 1. Evidencias del trabajo en equipo

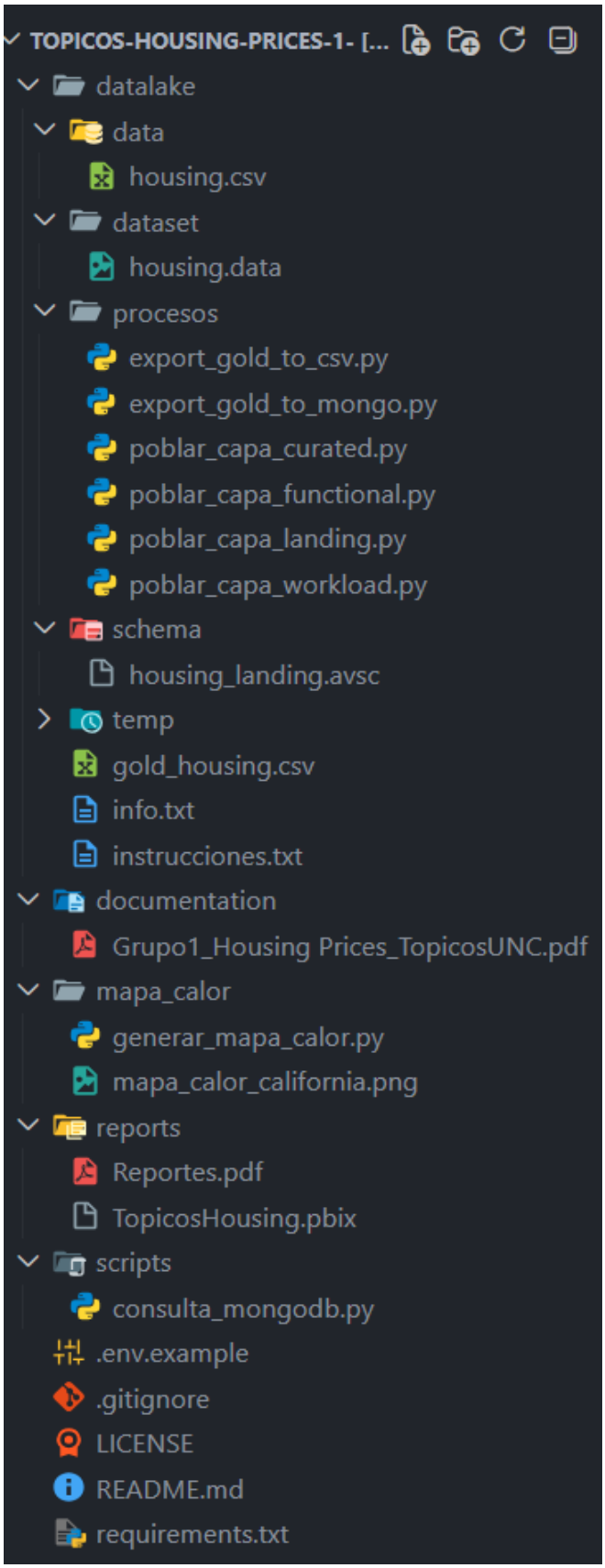
Foto Equipo Trabajo coherente con los integrantes que colocan en la caratula



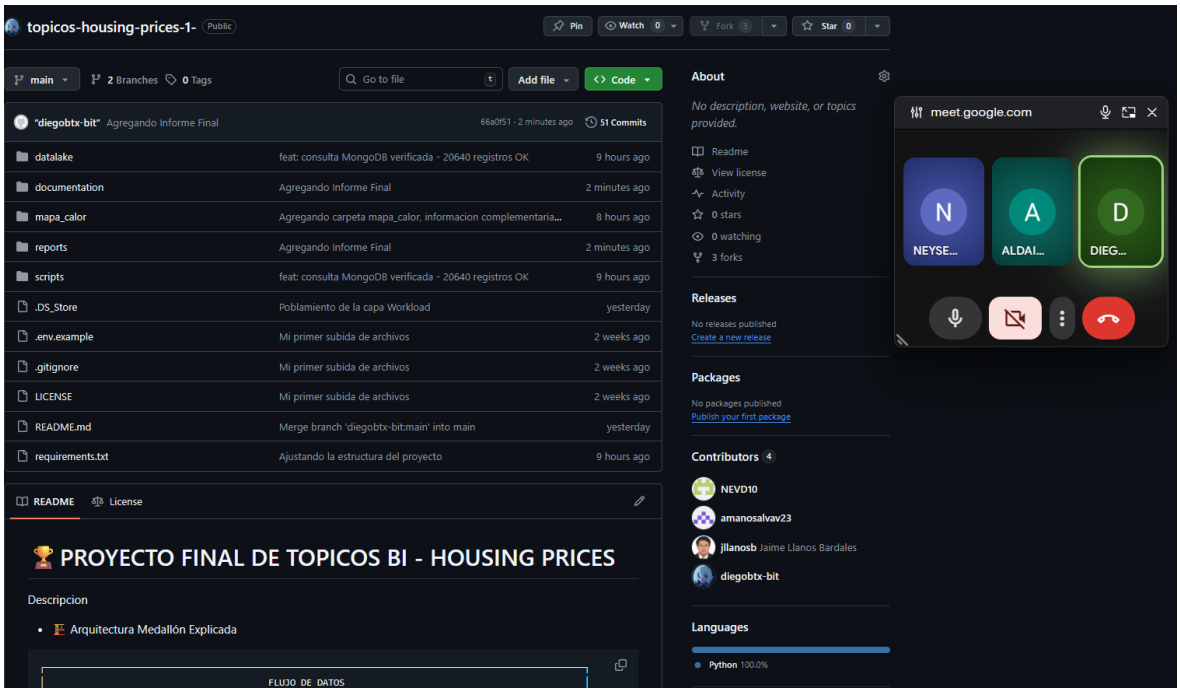
Historial de últimos commits, por parte del equipo de trabajo



ANEXO N° 2. Captura de la Estructura del Proyecto final en Vs Code.

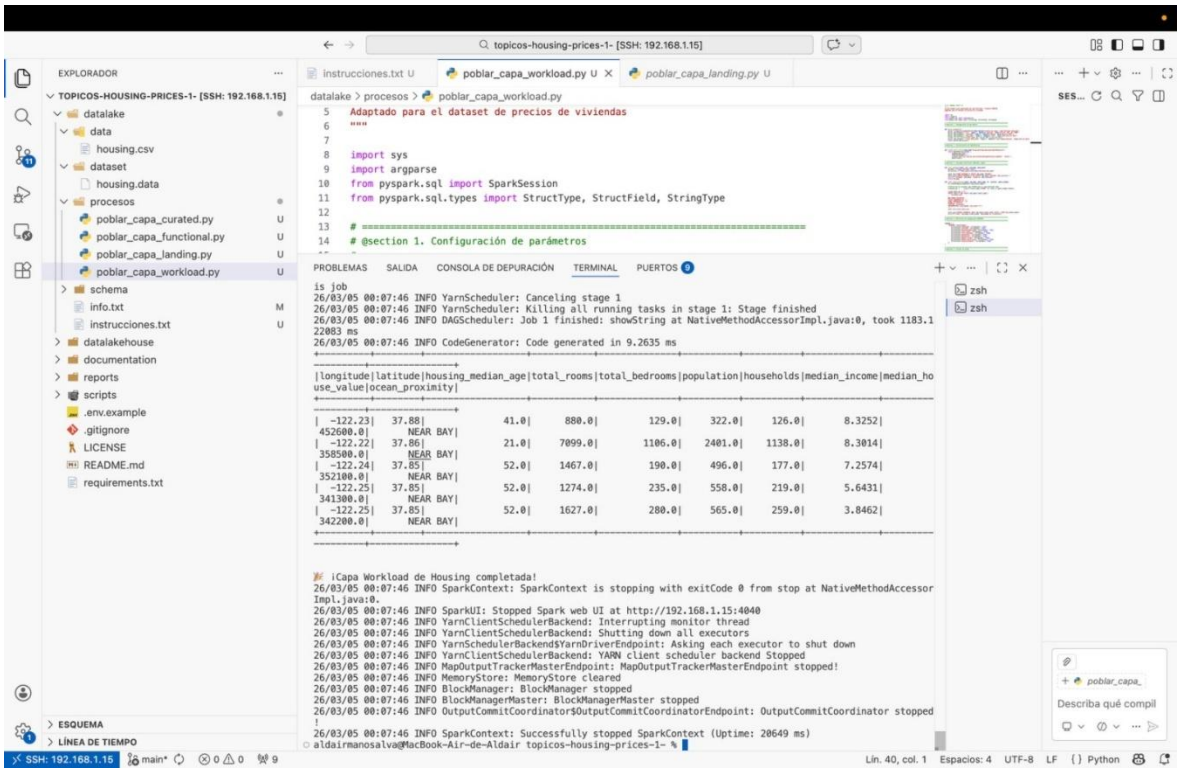


ANEXO N° 3. Captura de la Estructura del Proyecto final en Github.

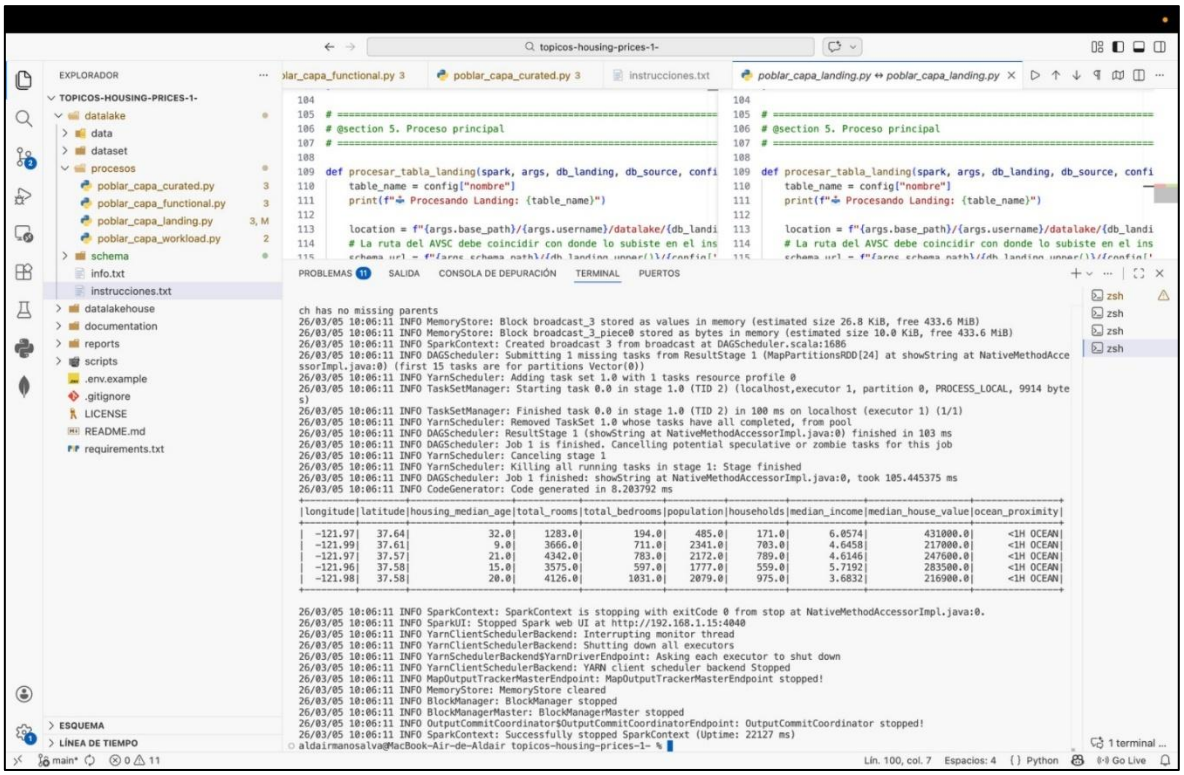


ANEXO N° 4. Capturas de ejecución PySpark (1 por capa workload, landing, curated, functional)

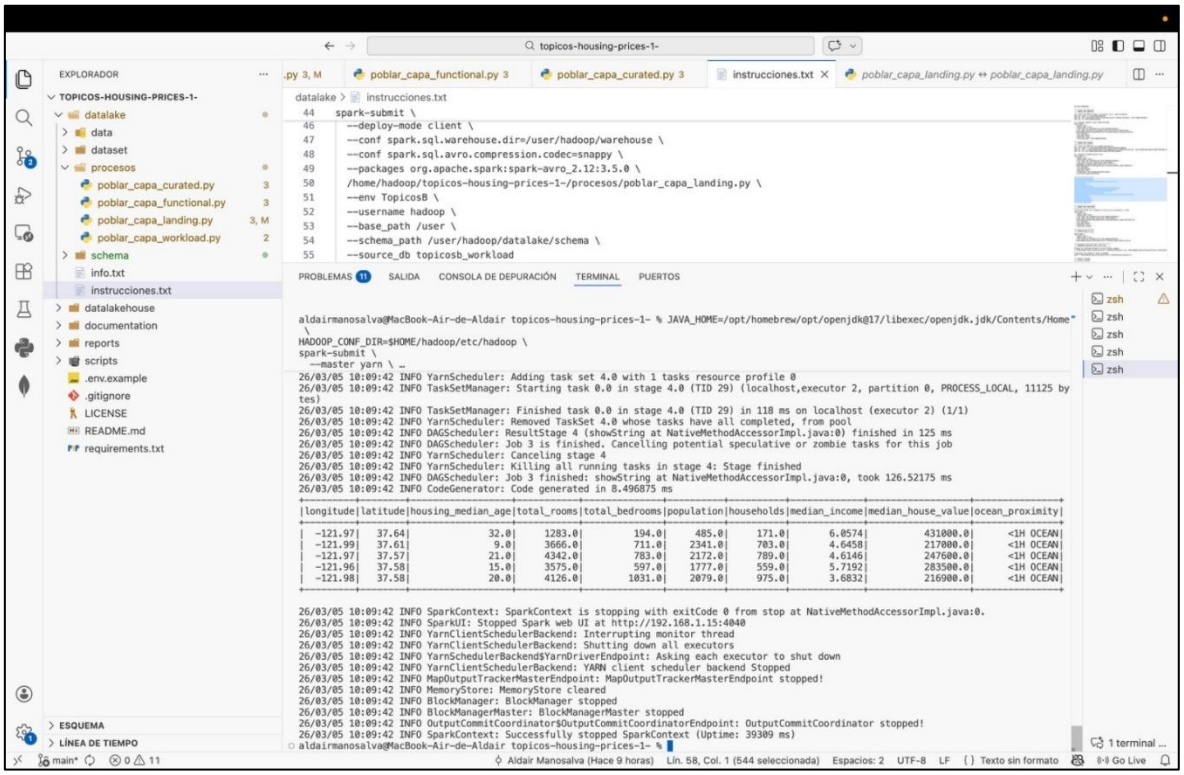
Capa_workload



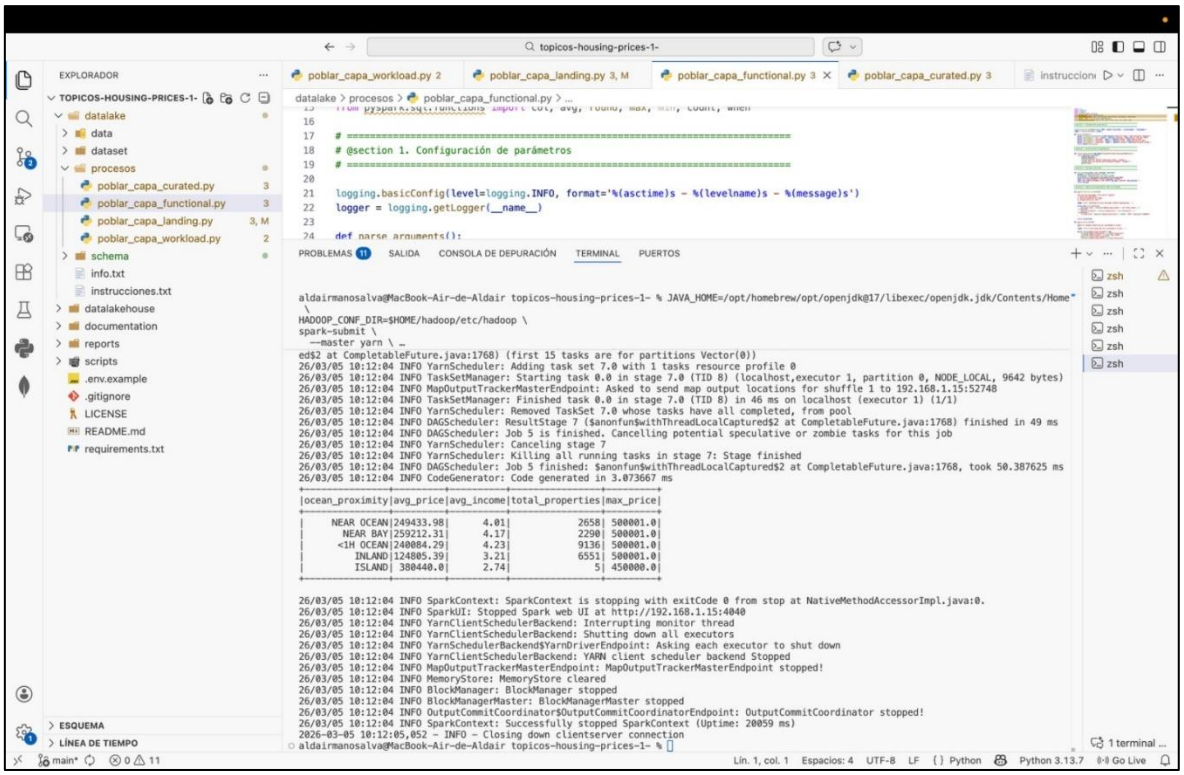
Capa_landing



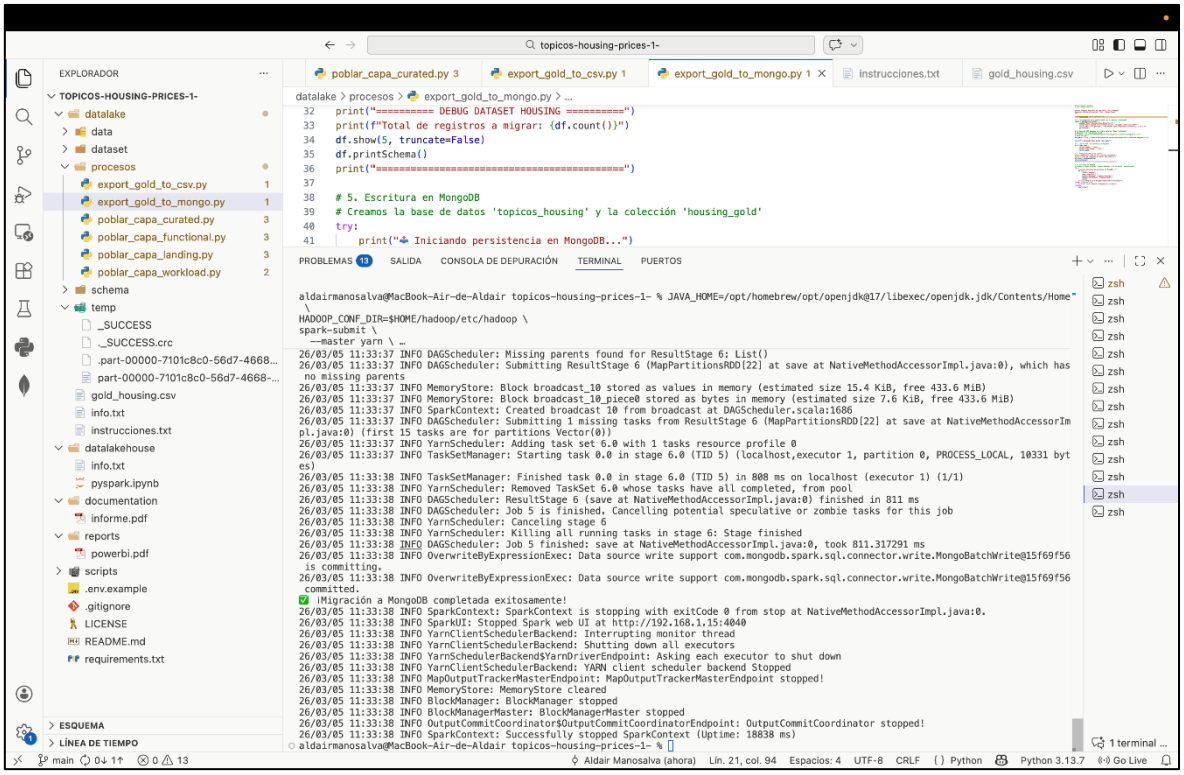
Capa_curated

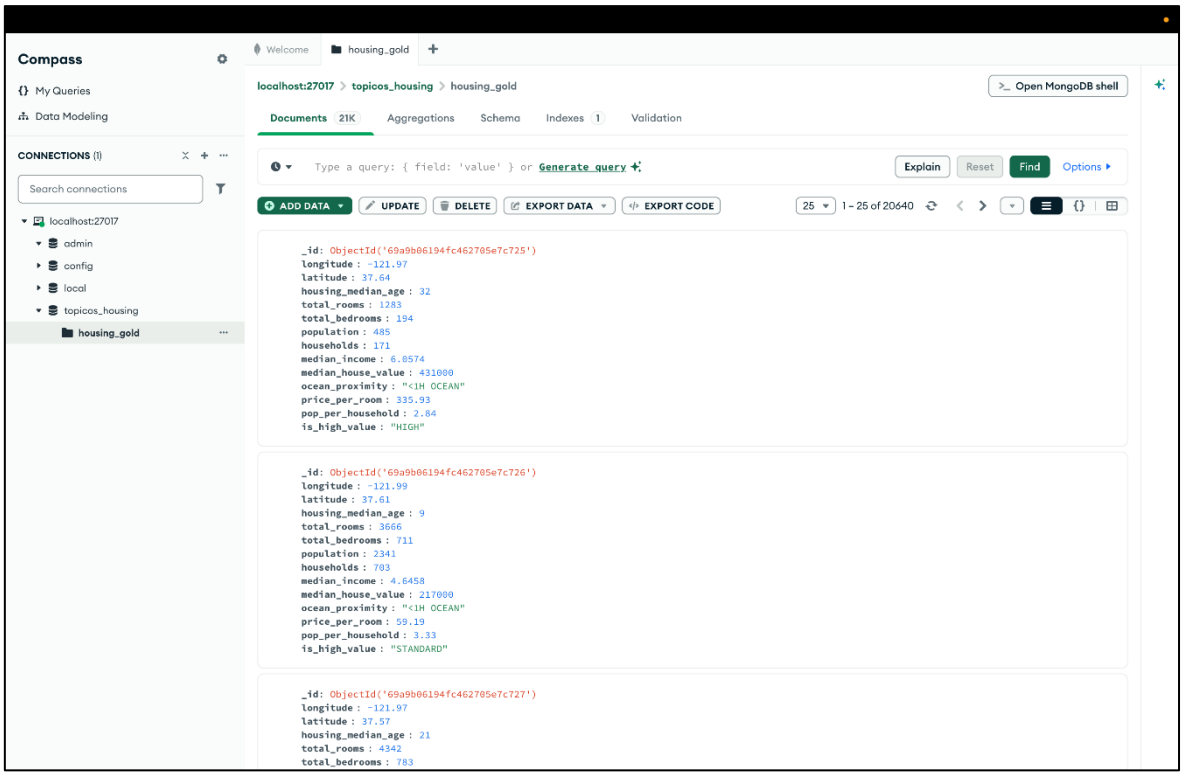
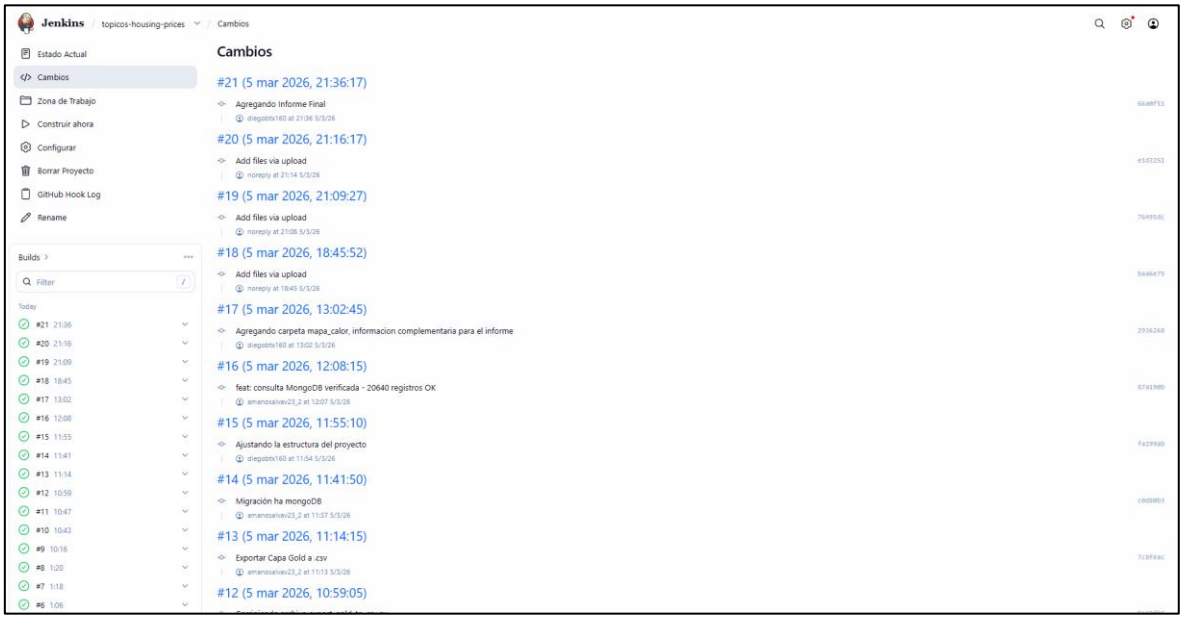


Capa_functional



ANEXO N° 5. Captura de la carga de datos de la capa functional a MongoDB (Local o Atlas)





ANEXO N° 6. Captura Tareas Automatizadas o manual de CI correctas en Jenkins